

Checkpoint de Avance Generador YAPar

Universidad del Valle de Guatemala · Diseño y Construcción de Compiladores · Reporte de progreso semanal

Equipo	Fernando Mendoza
Periodo del reporte	3 al 10 de mayo de 2026 (Sprint 1)
Repositorio	https://github.com/lfmendoza/compiladores-yapar
Estado general	En cronograma · sin bloqueos críticos
Fecha del reporte	10 de mayo de 2026

1. OKR de la fase de compilación

Objetivo: Entregar la fase de compilación YAPar con autómata LR(0) canónico, tabla SLR(1) y motor del parser funcionando sobre el conjunto de gramáticas de prueba acordadas, con documentación final completa antes de la fecha límite.

KR	Resultado clave	Meta	Estado
KR1	Arquitectura documentada y aprobada por el catedrático.	1 doc	✓ Hecho
KR2	Módulos del pipeline implementados con tests unitarios al 80% de cobertura mínima.	8 / 8	1 / 8
KR3	Parser ejecuta exitosamente sobre la batería de gramáticas de prueba.	3 / 3	0 / 3
KR4	Documentación final, video y entrega completa antes de la fecha límite.	1 entrega	En curso

Avance OKR global: 25%. La semana cerró con el KR1 completo y el KR2 en marcha. KR3 depende de tener al menos los módulos 1–6 funcionando, planeados para las próximas dos semanas.

2. Resumen del sprint

Esta semana cerramos la fase de diseño arquitectónico y abrimos la implementación. La estrategia fue paralelizar el trabajo: **Fernando** arrancó el módulo 1 (lector del archivo .yalp) que es la puerta de entrada al sistema; luego se empezó las estructuras de datos del módulo 3 (Symbol, Production, Grammar), que son la moneda de cambio entre todos los demás módulos; finalmente se hizo un spike de viabilidad con Graphviz para el módulo 9 y montó la infraestructura de CI básica del repo. La integración entre los tres frentes se hace cuando el módulo 1 alcance a producir un objeto Grammar válido proyectado para principios de la semana 2.

3. Tablero Kanban

El tablero refleja el estado al cierre del sprint 1. Las tarjetas en "Hecho" corresponden a los entregables del KR1 y la infraestructura mínima del repo. Las dos tarjetas activas tienen un único responsable cada una para evitar bloqueos por revisión cruzada en esta etapa temprana.

Tablero Kanban — Sprint 1 (3–10 mayo)

Repositorio: yapar-ir0 · Proyecto: Generador YAPar

POR HACER	EN PROGRESO	HECHO
4. FIRST / FOLLOW [Integrante 2] Alta · core 5. Autómata LR(0) [Integrante 3] Alta · core 6. Tabla SLR(1) Fernando Alta · core 7. Motor del parser Fernando Media · driver 8. Visualización Graphviz [Integrante 3] Baja · tooling Tests de integración E2E Todos Media · qa	1. Lector de .yalp Fernando Alta · parser-input 3. Repr. interna de gramática [Integrante 2] Alta · datastruct	Arquitectura del sistema Todos OKR-KR1 Diagrama de módulos (drawio) Fernando design Documento de diseño (PDF/DOCX) Todos OKR-KR1 Setup del repositorio Fernando infra Convenciones de Git y CI base [Integrante 3] infra Integración con YALex previo [Integrante 2] spike
Métricas del sprint: 6 tarjetas en Hecho · 2 en Progreso · 6 en backlog · Avance OKR: 25%		

4. Distribución de responsabilidades

Cada integrante es responsable de un subconjunto de módulos del pipeline. La asignación se hizo balanceando carga total, dependencias entre módulos y especialización previa de cada quien

Integrante	Módulos asignados	Foco esta semana	Entregable concreto
Fernando	1, 6, 7 (lector, tabla SLR, motor)	Parser del .yalp y validación de sintaxis del DSL.	PR #4: lector funcional para gramáticas simples.
Fernando	3, 4 (gramática, FIRST/FOLLOW)	Symbol/Production/Grammar + augmentación.	PR #5: estructuras + tests unitarios.
Fernando	5, 8 (autómata LR(0), visualización)	Spike de Graphviz + scaffolding del closure/goto.	PR #3 (merged): infra de CI · PR #6 (draft): renderer.

5. Changelog de commits

Extracto de `git log --oneline --all` del repositorio durante el período del reporte. Se siguen las convenciones de **Conventional Commits** (feat, fix, docs, chore, test) para que el changelog pueda generarse automáticamente al cierre del proyecto.

Hash	Autor	Fecha	Mensaje
a3f4b91	Fernando M	2026-05-10	feat(yalp-reader): parsing inicial de declaraciones de tokens
c91e2d8	Fernando M	2026-05-10	feat(grammar): augmentación con S' y numeración de producciones
7b5fa12	Fernando M	2026-05-09	feat(yalp-reader): tokenizador del DSL de YAPar (tokens, IGNORE)
e02c4d6	Fernando M	2026-05-09	chore(ci): pipeline de GitHub Actions para tests y lint
4d8a3f2	Fernando M	2026-05-08	feat(grammar): Symbol, Production y Grammar como dataclasses frozen
2a91e7b	Fernando M	2026-05-08	docs(viz): spike de Graphviz – render mínimo del autómata
9f3c1a4	Fernando	2026-05-07	docs(arch): diagrama de módulos en draw.io + export PNG
1e7b3d9	Fernando	2026-05-07	docs(arch): documento de arquitectura – versión final entregable
5c2e9a1	Fernando M	2026-05-06	test(yalex): integración con la suite YALex previa – fixtures
8d4f6b3	Fernando M	2026-05-05	chore(repo): CODEOWNERS y plantillas de PR / issue
3a1c8e5	Fernando	2026-05-04	chore(repo): scaffolding inicial, layout src/, .gitignore
0f9e2b7	Fernando	2026-05-03	chore(repo): commit inicial · README · licencia MIT

6. Riesgos identificados y mitigaciones

Riesgo	Impacto / Probabilidad	Mitigación
Gramáticas de prueba con conflictos shift/reduce no resolubles por SLR.	Alto / Media	El módulo 6 reporta cada conflicto con el ítem y el lookahead que lo provoca. Si aparecen, se simplifica la gramática o se escala a LALR(1) en una iteración posterior.
Bug en la deduplicación de estados rompe la terminación del closure/goto.	Alto / Baja	El <code>_index frozenset→id</code> se prueba con un caso patológico (gramática con ϵ -producciones). Test de regresión obligatorio en el módulo 5.
Desincronización con el formato de tokens del YALex previo.	Medio / Media	Se definió un contrato Token(tipo, lexema, línea, columna) en el documento de arquitectura. Test de integración con outputs reales del YALex existente.
Acumulación de trabajo al final del semestre por solapamiento con otros cursos.	Alto / Alta	Sprints de una semana con checkpoint público los viernes. Si un sprint cierra con más de 2 tarjetas atrasadas, se renegocia el alcance del siguiente.

7. Próximos pasos para Sprint 2

Plan tentativo para la semana del 13 al 17 de mayo. Cada bloque tiene un dueño explícito y un criterio de aceptación verificable.

Día	Tarea	Responsable	Aceptación
Lun 13	Cerrar módulo 1 (lector .yalp) y abrir PR.	Fernando M	PR + tests
Mar 14	Cerrar módulo 3 (gramática + augmentación) y mergear.	Fernando M	merge a main
Mié 15	Empezar módulo 4 (FIRST / FOLLOW) en pair-programming.	Fernando M	spike de 4h
Jue 16	Kick-off módulo 5 (autómata LR(0)).	Fernando M	rama feat/lr0
Vie 17	Demo interna: gramática mínima procesada end-to-end (sin tabla SLR aún).	Fernando M	demo OK